

TRACE-ROUTER: TASK-CONSISTENT AND ADAPTIVE ONLINE ROUTING FOR AGENTIC AI

Ritik Raj^{1,*}, Souvik Kundu^{2,*}, Sarbartha Banerjee¹, Dheemanth Joshi³, Ishita Vohra¹, Tushar Krishna¹

¹Georgia Institute of Technology ²Intel ³Texas A&M University

*Correspondence: ritik.raj@gatech.edu, souvikk.kundu@intel.com

ABSTRACT

Routing to select large language models (LLMs) with different cost–quality trade-offs has become a fundamental deployment feature of enterprise AI. Existing routers, primarily make independent routing decisions for each LLM call. However, agentic applications execute as long-horizon workflows whose quality is determined only by a delayed, task-level outcome. This mismatch prevents per-call routers from correctly attributing feedback to individual routing decisions. Towards mitigating this, we present TRACE-Router, a task-level routing framework that aligns routing with the unit of supervision. TRACE-Router assigns each task to a model once at admission using a contextual bandit, pins all subsequent LLM calls to the selected backend, and updates its policy using the task’s terminal reward, jointly accounting for accuracy and latency. By leveraging delayed task feedback, TRACE-Router learns routing policies that adapt to the workload while avoiding explicit task-complexity estimation. Across three agentic benchmarks, TRACE-Router consistently improves the accuracy–latency trade-off, achieving non-dominated Pareto frontier points. On τ^2 -Bench, it outperforms latency-matched interpolation between individual models by 7–8 accuracy points, while on Terminal-Bench it achieves 7.1 higher accuracy points than the strongest single model baseline with 36% lower latency.

1 INTRODUCTION

Modern LLM deployments operate over heterogeneous model pools, where smaller models provide low-latency, cost-efficient inference, while larger models deliver higher reliability on challenging tasks at significantly greater computational cost. No single model is uniformly optimal, and guessing wrong is costly in both directions—always serving the strongest model spends frontier-scale compute on instances a small model would have solved, while always serving the cheapest degrades accuracy precisely where accuracy matters. Routing has consequently become a central primitive for LLM deployment (Chen et al., 2024a; Ong et al., 2024; Guha et al., 2024; Chen et al., 2024b; Feng et al., 2025; Tsiourvas et al., 2026; Xue et al., 2026).

Nearly all of these routers inherit a request-level abstraction: *Given a prompt, which model should answer it?* This is the right granularity for single-turn inference and the wrong one for agentic workloads, where a task is not a request but a stateful execution trace of many LLM calls interleaved with tool use, retrieval, validation, and environment transitions. Each LLM call can be routed independently, and each decision may appear locally optimal, yet the resulting execution can be globally suboptimal. A seemingly simple intermediate turn may be routed to a smaller model, discarding the model-specific state accumulated earlier, while the cost of that decision is determined by the backend’s ability to complete the entire task rather than the difficulty of the individual call. For example, in the telecom domain of τ^2 -Bench Barres et al. (2025), the smaller backend solves only 10.4% of tasks compared to 60.2% for the larger backend. Consequently, a single mid-task downgrade to the smaller model can cause an otherwise successful execution to fail. More fundamentally, task success or failure is observed only at termination, producing a single delayed outcome that must be attributed to all routing decisions made throughout the execution. Without a principled mechanism for credit assignment, the router cannot effectively learn from task-level feedback.

This observation motivates a simple principle: *The unit of routing should match the unit of feedback*. In agentic workloads, feedback is inherently task-level—a program either passes its tests, an environment goal is achieved, or a judge assigns a final score—and a decision that persists over the entire trace is the finest-grained choice such a signal can credit unambiguously.

We introduce TRACE-Router, a framework that acts on this principle through online model selection at task-level granularity. Prior routers predict a model from the prompt using a classifier trained offline on labeled routing data (Ong et al., 2024; Feng et al., 2025; Tsiourvas et al., 2026), which fixes the routing policy before deployment and cannot correct a mistaken assignment once the workload reveals it. TRACE-Router instead learns which backend to use from the outcomes of completed tasks. It places each task into a coarse difficulty class from features available at admission, then treats model choice within that class as a bandit problem, exploring the pool over the stream of tasks and converging on the backend that best trades accuracy against latency for tasks of that kind. The policy therefore requires no offline training and adapts as the workload runs.

Concretely, a new task is assigned a coarse routing context at admission, and the contextual bandit for that context selects one backend. A persistent task identifier, attached by the agent harness to every request, pins all subsequent calls in that task to the selected backend. On termination, graded accuracy and end-to-end latency are scalarized by a preference α into a single delayed reward that updates only the bandit that made the decision. The shift in granularity mirrors a broader move in agent serving toward treating workflows and programs, rather than requests, as first-class scheduling objects (Kang et al., 2026; Luo et al., 2025; Wadlom et al., 2026; Wagenländer et al., 2026); TRACE-Router further differs from trajectory-level routers that commit one model per trace using an offline-trained value model (Son et al., 2026) in learning its policy online, from task outcomes alone.

Figure 1 showcases how the variation of the reward parameter α enables TRACE-Router to learn a family of routing policies that trade accuracy for latency. Across the two τ^2 -Bench domains, these policies occupy the empirical Pareto frontier alongside the two single-model endpoints, while all heuristic and learned baselines are dominated. Importantly, merely appearing on the frontier is not sufficient, as a random mixture of two endpoint models also traces the line segment between them in expectation. The appropriate comparison is therefore a latency-matched random mixture, against which TRACE-Router demonstrates that adaptive task-level routing yields better accuracy than probabilistic model mixing at the same latency budget. TRACE-Router exceeds it by 7.5, 7.1, and 7.9 points at its three operating points, reaching 61.2% task accuracy at 26.4 s—67% of the latency separating the two backends for 90% of the accuracy separating them—and is 27.8 points more accurate than the strongest baseline at identical latency. Nor is the advantage confined to approaching the larger backend from below: on Terminal-Bench, where the two backends resolve substantially different task subsets, TRACE-Router resolves 46.8% of tasks at 172 s against 39.7% at 268 s for always serving the larger backend—7.1 points more accurate at 36% lower latency, and within 1.2 points of a task-matched accuracy oracle.

Our contributions are as follows:

- **Task-level routing for agentic workloads.** We formulate model routing as *task-consistent online model selection*, where a single backend is assigned to an entire execution trace and updated using the task’s terminal outcome. By aligning the routing decision with the unit of feedback, our formulation enables principled credit assignment from delayed task-level rewards.

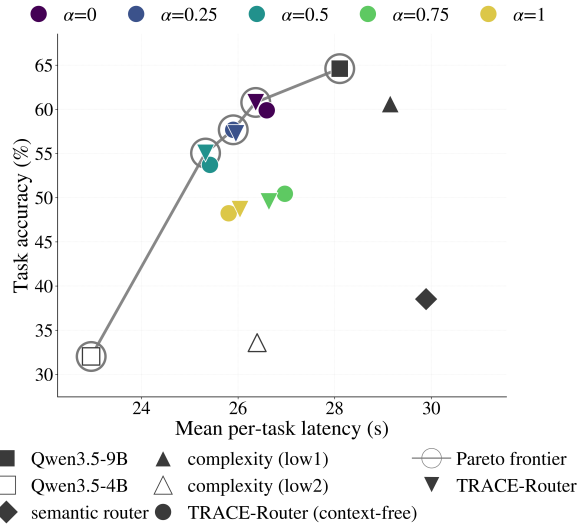


Figure 1: **Task accuracy-latency trade-off on τ^2 -Bench**, averaged over the retail and telecom domains. Every interior frontier point is produced by TRACE-Router. Every interior frontier point is produced by TRACE-Router. Every interior frontier point is produced by TRACE-Router. Every interior frontier point is produced by TRACE-Router. Every interior frontier point is produced by TRACE-Router.

- **Context-aware online routing.** We introduce TRACE-Router, a contextual bandit-based router that maintains independent routing policies for coarse task contexts. This design enables the router to adapt to workload regimes with distinct model performance characteristics and learn context-specific accuracy–latency trade-offs online.
- **Improved accuracy–latency trade-offs.** Across four agentic benchmarks and two backend pairs, TRACE-Router consistently learns routing policies on the empirical accuracy–latency Pareto frontier, outperforming heuristic and learned baselines. It exceeds latency-matched model interpolation by **7–8 accuracy points** on τ^2 -Bench and strictly dominates the larger backend on Terminal-Bench while reducing latency.

2 RELATED WORK

Request-level LLM routing. A growing line of work studies how to select among multiple LLMs for each request in order to improve cost–quality trade-offs. Early systems use cascades or defer-to-stronger-model policies (Chen et al., 2024a; Aggarwal et al., 2024), while later approaches learn request-level routers from supervision, preference data, unsupervised agreement signals, graph structure, or observational feedback (Ong et al., 2024; Guha et al., 2024; Chen et al., 2024b; Feng et al., 2025; Tsiourvas et al., 2026; Xue et al., 2026; Zhang et al., 2026). These methods differ in training signal and model class, but they share the same basic abstraction: a routing decision is attached to an individual prompt or query. Our work differs in granularity. We route *task traces*, not isolated requests, and learn from delayed task-level feedback rather than only from per-request labels or scores.

Routing with richer intermediate signals. Several recent works enrich routing with signals beyond the input prompt alone. Collaborative or reasoning-time methods use confidence estimates, hidden-state diagnostics, predicted latent outputs, or partial generations to decide when to escalate from a smaller model to a larger one (Zheng et al., 2025; Huang et al., 2026; Tian et al., 2026). The closest recent work is SWE-Router, which shows that in multi-turn software-engineering tasks, conditioning on a short partial trajectory can outperform routing from the task description alone (Son et al., 2026). Our setting is complementary as we do not assume a fixed two-stage weak-to-strong escalation pattern or a domain-specific exploratory prefix. Instead, we choose one model for the entire task trace, bind the trace to that model via a sticky key, and update the routing policy from delayed task-level reward.

Program-aware serving for agentic workloads. A concurrent systems literature argues that agentic workflows should be treated as first-class programs rather than as unrelated inference calls. Recent serving systems model workflows or programs explicitly to improve scheduling, cache reuse, and end-to-end latency (Kang et al., 2026; Luo et al., 2025; Wadlom et al., 2026; Wagenländer et al., 2026). These works are highly aligned with our problem framing: They show that request-level abstractions are too weak for agentic execution. Our contribution is complementary as we focus on *online model selection* at the task level, whereas program-aware serving systems focus on scheduling and resource management once a program is already executing.

Online routing under partial feedback. Our work is also related to recent routing methods that move beyond full information and learn from deployed outcomes only (Tsiourvas et al., 2026). The key additional challenge in our setting is that the reward is both *delayed* and *task-scoped*: the relevant outcome is observed only after a multi-call trace completes, and it must be attributed to the single routing decision that governed the entire trace. This requires a different decision object—the task trace itself—and a sticky mechanism for maintaining that decision throughout execution.

3 METHODOLOGY

Most LLM routers select a model independently for each request. This abstraction is natural for single-turn inference, but it is misaligned with agentic execution, where one task may issue multiple LLM requests interleaved with tool use, retrieval, validation, memory updates, and environment transitions. Independent request-level decisions can switch models midway through a task, fragment

credit assignment across several local decisions, and invalidate model-specific state accumulated earlier in the trace.

TRACE-Router instead treats the *task trace* as the unit of both routing and learning. As illustrated in Figure 2, the router makes one model-selection decision when a task first appears, stores the resulting task-to-model binding, and reuses it throughout the trace. Once the task terminates, its final accuracy and end-to-end latency produce a delayed reward for the policy that made the original decision. The framework combines task-consistent routing (subsection 3.1), context-conditioned model selection (subsection 3.2), an online bandit policy over backends (subsection 3.3), and delayed task-level feedback (subsection 3.5).

TRACE-Router is specified independently of any particular agent harness, context classifier, evaluator, or serving stack. It requires four things: a pool of candidate backends; a task identifier that persists across the requests of one trace; a context signal available before the first routing decision; and a graded task-level outcome observable after termination. Everything else—how contexts are computed, how accuracy is graded, how many backends the pool contains—is a deployment choice. We fix those choices in subsection 4.1 and vary them in the ablations of subsection 4.5 and subsection 4.6.

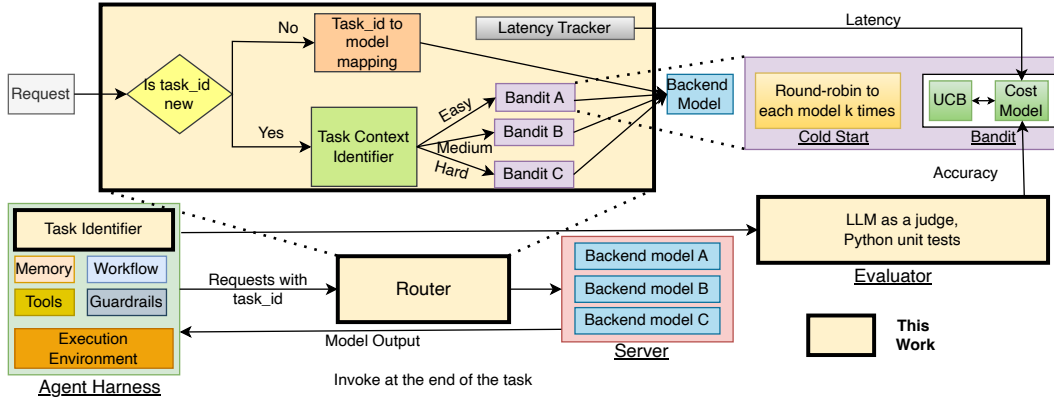


Figure 2: **TRACE-Router overview.** Each request carries a persistent task identifier. Requests from an active task reuse its stored backend assignment, while a new task is assigned a coarse context and routed by the corresponding bandit; the figure illustrates a three-context instantiation. After task completion, graded accuracy and end-to-end latency are combined into a delayed reward that updates the bandit responsible for the routing decision.

3.1 TASK-CONSISTENT ROUTING

Let $\mathcal{M} = \{1, \dots, K\}$ be a pool of K candidate LLM backends, indexed by m . A task t induces an execution trace

$$\Pi_t = (q_{t,1}, q_{t,2}, \dots, q_{t,H_t}), \quad (1)$$

where $q_{t,h}$ is the h -th routed LLM request issued during the task and H_t is the total number of such requests. Tool calls, host-side computation, retrieval, and environment interactions may occur between consecutive requests, and H_t is not known when the task is admitted.

The agent harness attaches a persistent task identifier κ_t to every request in Π_t . The router maintains an *active-task table* $\mathcal{A}$, a mapping from each identifier to the routing decision made for its task, which we write as a record with a context field and a model field,

$$\mathcal{A} : \kappa_t \mapsto (\text{ctx} = c_t, \text{mdl} = m_t), \quad (2)$$

where c_t is the task’s routing context (subsection 3.2) and $m_t \in \mathcal{M}$ is the backend selected for it.

For an incoming request the router first checks whether κ_t is already a key of $\mathcal{A}$, i.e. $\kappa_t \in \text{dom}(\mathcal{A})$. If so, the request belongs to a task already in progress, and the router reuses the stored backend by reading its model field, $\mathcal{A}(\kappa_t).\text{mdl}$:

$$m_t = \mathcal{A}(\kappa_t).\text{mdl}. \quad (3)$$

Neither context identification nor model selection is repeated: only the first request of a task invokes the routing policy. When a task is seen for the first time, the router selects a backend (subsection 3.3) and records the decision,

$$\mathcal{A}(\kappa_t) \leftarrow (\text{ctx}=c_t, \text{mdl}=m_t), \quad (4)$$

after which every subsequent request in the trace reuses this assignment through Equation 3.

This binding yields a single credit-assignment target: the final task reward is attributed to the one backend that served the entire trace. It also preserves model identity across the task, preventing cross-model invalidation of state accumulated during execution. Actual state reuse remains dependent on the capabilities and runtime state of the serving backend; task consistency is a necessary condition for it, not a guarantee of it.

3.2 CONTEXT-CONDITIONED MODEL SELECTION

A global routing policy pools feedback across tasks that may induce conflicting model preferences. Tasks for which latency dominates may favor smaller models, whereas tasks for which correctness dominates may favor stronger models. When a workload contains both regimes, a single policy learns a value for each backend that averages over them and is correct for neither; the effect is large enough to determine which operating points are reachable at all (subsection 4.3). We therefore expose coarse workload structure through a discrete task context.

Let x_t be an initial descriptor of task t —the information available at admission, before any LLM call—and let g be a context function mapping it to one of a finite set $\mathcal{C}$ of routing contexts,

$$c_t = g(x_t), \quad c_t \in \mathcal{C}. \quad (5)$$

The framework imposes no particular realization of g . It requires only that c_t be computable from information available before the first routing decision—so that no LLM call is needed to route one—and that $|\mathcal{C}|$, the number of contexts, be small enough for each context to accumulate feedback over the task stream. These two requirements pull against each other: a finer partition separates regimes more sharply but divides the stream further, and $|\mathcal{C}|$ also scales the initialization cost of subsection 3.3. Any classifier meeting them can be substituted; subsection 4.1 states the one we use.

For each context $c \in \mathcal{C}$, TRACE-Router maintains an independent bandit B_c , an instance of the online policy of subsection 3.3 with its own per-arm statistics. Let π_c denote the selection distribution that B_c induces over backends. A new task in context c_t is routed by sampling

$$m_t \sim \pi_{c_t}(\cdot), \quad (6)$$

after which (c_t, m_t) is stored in $\mathcal{A}$ via Equation 4. Feedback from task t updates only B_{c_t} . Context conditioning therefore changes the state of the learner, rather than merely rescaling the reward of a shared global policy.

3.3 COLD-START CONTEXTUAL UCB

Partitioning a finite task stream across contexts appears to create a statistical difficulty: each context receives only a fraction of the observations, so a cold-start bandit must spend its first tasks in every context on uninformed exploration. We initially treated this as the binding constraint and designed a prior-warm-started policy around it (Appendix A). The ablation in subsection 4.4 does not support that design: warm-starting matches cold-start accuracy while systematically shifting selection toward slower models. TRACE-Router’s default policy is therefore cold-start contextual UCB, and the warm-started variant is retained as an ablation.

Each bandit B_c maintains, for every backend $m \in \mathcal{M}$, a pull count $N_{c,m}$ (the number of completed tasks in context c that were served by m) and a reward sum $S_{c,m}$ (the total reward those tasks returned), both initialized to zero. To give every arm a defined estimate before the policy becomes selective, the first $k|\mathcal{M}|$ tasks arriving in context c are assigned round-robin, so that each arm receives $k \geq 1$ forced pulls. Initialization therefore consumes

$$|\mathcal{C}| \cdot k|\mathcal{M}| \quad (7)$$

tasks in total, linear in both the number of contexts $|\mathcal{C}|$ and the pool size $|\mathcal{M}|$. Thereafter the empirical value estimate of backend m in context c is the mean reward it has returned,

$$\hat{\mu}_{c,m} = \frac{S_{c,m}}{N_{c,m}}, \quad (8)$$

and for a new task assigned to context c_t the router selects the arm maximizing an upper confidence bound,

$$m_t = \arg \max_{m \in \mathcal{M}} \left[\hat{\mu}_{c_t, m} + \sqrt{\frac{2 \log(1/\delta)}{N_{c_t, m}}} \right], \quad (9)$$

where the second term is an exploration bonus and $\delta \in (0, 1)$ is a fixed confidence parameter. This selection rule is the distribution π_{c_t} referenced in Equation 6.

The confidence radius in Equation 9 depends on the arm's own count $N_{c_t, m}$ and on δ , but not on the total number of tasks seen in the context. Here δ therefore acts as a fixed exploration strength rather than as a failure probability in an anytime bound: a smaller δ widens every radius uniformly and lengthens exploration, a larger δ commits sooner. We make no asymptotic regret claim for this variant. UCB is itself one choice of policy for this slot; subsection 4.5 compares it against ε -greedy and Thompson sampling and reports the sensitivity of each to its exploration parameter.

3.4 TASK-LEVEL ACCURACY-LATENCY REWARD

The routing decision is evaluated using the outcome of the complete task. Let $a_{t, m} \in [0, 1]$ be the graded accuracy of task t when served by backend m , and let $\ell_{t, m} \geq 0$ be its end-to-end execution latency in seconds. We normalize latency against a workload-scale constant $\ell_0 > 0$ and clip it to the unit interval,

$$\tilde{\ell}_{t, m} = \min\left(\frac{\ell_{t, m}}{\ell_0}, 1\right), \quad (10)$$

where ℓ_0 is chosen so that typical task latencies fall below the clipping threshold. For a trade-off parameter $\alpha \in [0, 1]$, the task reward combines accuracy against normalized latency,

$$r_{t, m}^{(\alpha)} = (1 - \alpha) a_{t, m} - \alpha \tilde{\ell}_{t, m}. \quad (11)$$

At $\alpha = 0$ the policy optimizes accuracy alone; at $\alpha = 1$ maximizing reward is equivalent to minimizing clipped latency; intermediate values encode different preferences. For fixed α the reward lies in an interval of width one,

$$r_{t, m}^{(\alpha)} \in [-\alpha, 1 - \alpha], \quad (12)$$

a fixed-width scale on which the prior means of Appendix A are also expressed. The interval's location shifts with α , so rewards obtained under different preferences are not mutually comparable; accordingly, each value of α maintains an independent policy state, and observations are never shared across scalarizations. Sweeping α therefore traces a family of deployment policies rather than reweighting a single learned one.

The accuracy function is application-defined. It may be obtained from an exact validator, an execution-based evaluator, an environment-level success signal, or a learned judge. The framework requires only one graded score for the completed task.

3.5 DELAYED CREDIT ASSIGNMENT

Neither final accuracy nor total latency is available when the first request is routed. The reward is therefore observed only after task completion.

Once task t terminates, the evaluator returns its accuracy a_{t, m_t} and the latency tracker its latency ℓ_{t, m_t} ; these combine through Equation 11 into the realized reward $r_{t, m_t}^{(\alpha)}$. Using the task identifier κ_t , the router recovers the context and backend recorded for the task,

$$(c_t, m_t) = (\mathcal{A}(\kappa_t).ctx, \mathcal{A}(\kappa_t).mdl), \quad (13)$$

and updates only the corresponding context-backend statistics,

$$N_{c_t, m_t} \leftarrow N_{c_t, m_t} + 1, \quad S_{c_t, m_t} \leftarrow S_{c_t, m_t} + r_{t, m_t}^{(\alpha)}. \quad (14)$$

All other bandit states remain unchanged. The completed task is then removed from the active-task table,

$$\mathcal{A} \leftarrow \mathcal{A} \setminus \{\kappa_t\}. \quad (15)$$

For concurrent workloads, each in-flight task retains the backend selected at its arrival. Updates are applied in completion order, and new tasks use the statistics available from tasks that have already completed. Delayed and out-of-order feedback therefore affects future routing decisions without changing the backend assignment of an active task—the property that makes task-consistent routing compatible with a concurrent serving stack rather than requiring a serialized task stream.

4 EVALUATION

We ask whether task-consistent online routing improves the measured accuracy–latency frontier (subsection 4.2), whether context conditioning contributes beyond task consistency (subsection 4.3), and whether the policy’s two design choices—cold-start initialization (subsection 4.4) and UCB exploration (subsection 4.5)—are the right ones.

4.1 SETUP

TRACE-Router is implemented as a routing policy inside a patched LiteLLM proxy. All frontier results in subsection 4.2–subsection 4.4 are measured from live end-to-end runs; latency is wall-clock time per task at the proxy, inclusive of tool execution and environment interaction. Each benchmark (Table 1) pairs a smaller and larger backend from one family, and task counts are matched across strategies. Figure 3 shows three benchmarks; the τ^2 -retail panel appears in the appendix, and the retail–telecom average is the teaser of Figure 1. τ^2 -Bench airline (50 tasks) follows the same protocol but is too small to place on a frontier. LiveCodeBench uses difficulty-weighted pass@1 with weights $w_d = 1 - \bar{p}_d$ derived from the single-model baselines alone.

Table 1: Benchmark configurations. ℓ_0 is the latency normalizer of subsection 3.4, chosen so that typical task latencies fall well below the clipping threshold.

Benchmark	Smaller / larger backend	Tasks	Accuracy metric	ℓ_0
τ^2 -Bench (retail)	Qwen3.5-4B / 9B	114	task accuracy	60 s
τ^2 -Bench (telecom)	Qwen3.5-4B / 9B	114	task accuracy	60 s
LiveCodeBench	Qwen3.5-4B / 9B	300	difficulty-weighted pass@1	10 s
Terminal-Bench	Qwen3.5-9B / 27B-FP8	48	resolved rate	600 s

Context instantiation. We instantiate the context function g of subsection 3.2 as a regex-based complexity classifier over the initial task descriptor, assigning each task to one of three tiers, $\mathcal{C} = \{\text{EASY}, \text{MEDIUM}, \text{HARD}\}$, from keyword patterns and length thresholds. The choice is deliberately minimal: it needs no model call, no embedding, and no training data, and adds no measurable latency at admission. It is also exactly the classifier used by the complexity-router baseline, which is what makes that baseline the controlled comparison for our central claim—it observes the same context signal TRACE-Router observes and differs only in mapping tiers to models by a fixed rule rather than learning the map from task outcomes. Any classifier meeting the requirements of subsection 3.2 could be substituted, so our results should be read as attainable with a context signal of this quality, not as a ceiling.

Router and baselines. TRACE-Router uses cold-start contextual UCB with $k = 1$ forced pull per arm, $\delta = 0.1$, and sticky per-task assignment, sweeping $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$ with independent policy state per α ; with three contexts and two backends, initialization consumes six tasks. We evaluate a *context-free* variant (one global bandit) against the *context-conditioned* default. Baselines are the two single models; a *semantic router* assigning tasks by embedding similarity to per-route exemplars; and the complexity router above, at two split points. Two task-matched oracles—routing each task to a model that solves it, or to the faster model—are plotted as reference ceilings and excluded from the frontier; at temperature 0.7 these are stochastic references rather than bounds.

4.2 ACCURACY–LATENCY FRONTIERS

TRACE-Router holds the interior of the frontier. Averaged over the two τ^2 domains (Figure 1), the empirical Pareto frontier consists of the two single-model endpoints and three TRACE-Router

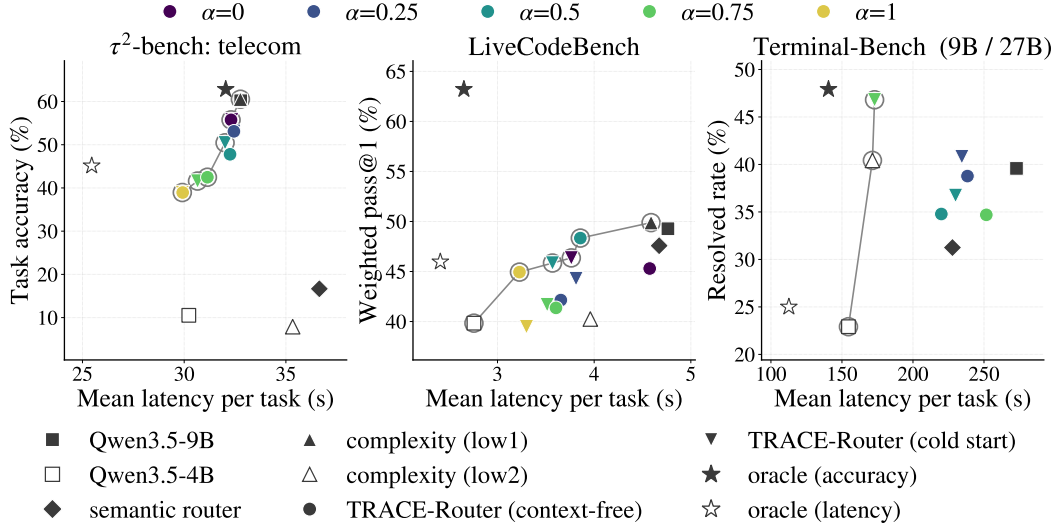


Figure 3: **Accuracy–latency frontiers.** Marker shape denotes method, color denotes preference α ; non-swept baselines are gray. Rings mark the non-dominated set and the connecting line the empirical Pareto frontier. Stars are task-matched oracles, excluded from the frontier. TRACE-Router holds interior frontier positions on all three benchmarks, dominates the smaller backend outright on telecom, and on Terminal-Bench holds a position strictly above the larger backend.

operating points—55.2%, 57.6%, and 61.2% at 25.4, 25.9, and 26.4 s—with every learned and heuristic baseline dominated. The pattern holds per benchmark. On LiveCodeBench the frontier runs from the small backend (2.75 s / 39.7%) through three TRACE-Router points to the complexity router at its upper split (4.6 s / 49.8%), which edges out always serving the larger backend (4.75 s / 49.3%): TRACE-Router holds every interior position, while the accuracy endpoint is held by a heuristic that is itself no faster than the large model. The semantic router is dominated on all three panels, in two of them by a wide margin.

The gains exceed latency-matched interpolation. Frontier occupancy alone is weak, since any mixture of two models traces the segment between them in expectation. The correct reference is a latency-matched random mixture, which on the τ^2 average predicts 47.3%, 50.5%, and 53.7% at TRACE-Router’s three operating points; TRACE-Router attains 55.2%, 57.6%, and 61.2%, exceeding interpolation by 7.9, 7.1, and 7.5 points. Stated as budgets, the $\alpha = 0$ policy spends 67% of the latency separating the two backends to capture 90% of the accuracy separating them, and is 27.8 points more accurate than the complexity router at identical latency.

A weaker backend is not reliably a faster one. The telecom panel shows the limiting case. The small backend resolves 10.5% of tasks at 30.2 s against the large backend’s 60.5% at 32.7 s: it is 49.9 points less accurate while saving only 2.5 s. It does not appear on the telecom frontier at all, because TRACE-Router at $\alpha = 1$ —where the reward weights latency alone—reaches 39.0% at 30.0 s, both faster and 28.5 points more accurate. A backend that cannot solve a task does not terminate early but consumes turns until the episode budget is exhausted, so end-to-end latency is governed substantially by whether the trace succeeds. We report this latency–competence coupling as measured; the turn-exhaustion mechanism is its most plausible cause but we do not instrument per-task turn counts to confirm it. It is also what creates room above the interpolation line: avoiding a backend on tasks it will fail recovers accuracy *and* the latency those failures would have consumed.

On Terminal-Bench, routing dominates both backends. The effect is strongest where the backends resolve the most dissimilar subsets. On Terminal-Bench, TRACE-Router at $\alpha = 0.75$ resolves 46.8% of tasks at 172 s against 39.7% at 270 s for always serving the 27B backend—7.1 points more accurate at 36% lower latency, and within 1.2 points of the accuracy oracle (48.0%). Routing here is not a concession to a budget but strictly better than either backend alone. Terminal-Bench’s 48

matched tasks make one resolved task worth 2.1 points, so the 7.1-point margin is between three and four tasks; we read the direction of this result as robust to that granularity and its magnitude as indicative.

4.3 CONTEXT CONDITIONING

Comparing the context-free and context-conditioned variants isolates partitioning from task consistency, since both are task-consistent. The aggregate difference is modest—+1.3 points at $\alpha = 0$ and +1.5 at $\alpha = 0.5$ on the τ^2 average, within noise elsewhere—but concentrated where it matters: the context-conditioned variant holds the frontier at $\alpha = 0$ and $\alpha = 0.5$ while the context-free variant is dominated at both. Partitioning does not raise the average operating point so much as determine which points are reachable. The mechanism is the spread across domains: a pooled policy must reconcile a regime where the small backend reaches 53.6% (retail, appendix) with one where it reaches 10.5%, learning a value for the small arm that is correct for neither. Since our context function is a regex classifier requiring no model call and no training data, these gains are a lower bound on what context conditioning can provide.

4.4 COLD START VERSUS WARM START

The short-stream argument of subsection 3.3 predicts that warm-starting should help. Figure 4 does not confirm it. The warm-started variant (Appendix A) initializes each context-backend pair with w pseudo-observations at a prior mean, making the prior policy’s preferred backend the initial argmax; after n real observations the prior contributes a fraction $w/(w+n)$ of the estimate, and $w \rightarrow 0$ recovers cold start.

Accuracy is indistinguishable on two of three benchmarks—60.0% against 60.8% on retail, 43.5% against 43.4% on LiveCodeBench—both well inside the spread across α . Terminal-Bench favors warm start by 3.8 points, a margin worth under two tasks at that benchmark’s 48, and whose $\alpha = 0.25$ entry uses the conservative seed of a replication pair; we do not read it as support.

Latency is unambiguous: warm start occupies a higher latency position on all three benchmarks (0.64 $\rightarrow$ 0.97 on retail, 0.42 $\rightarrow$ 0.45 on LiveCodeBench, 0.50 $\rightarrow$ 0.59 on Terminal-Bench). The retail figure is diagnostic. A position of 0.97 means the warm-started router behaves almost exactly like always serving the large backend, averaged over the entire sweep including values of α that weight latency heavily. The preferred model is selected and confirmed while alternatives accumulate real observations slowly: the prior is not overturned but self-reinforcing. Two properties explain the absence of the predicted benefit—with three tiers and two backends, cold start spends six tasks on forced exploration out of a stream of hundreds, and the pseudo-count mechanism buys early competence by suppressing exactly the exploration that discovers complementarity, which subsection 4.2 shows is where the gains are. TRACE-Router therefore defaults to cold start; warm-starting earns its complexity only when a handful of forced pulls is a material fraction of the stream, or the prior is reliable enough that suppressed exploration costs nothing.

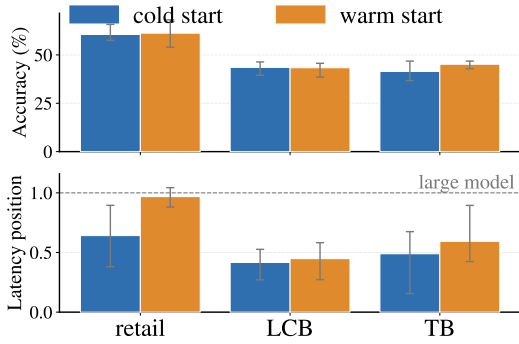


Figure 4: **Cold versus warm start.** Bars are the mean over the α sweep; latency normalized to the single-model gap, so 0 is as fast as the small backend and 1 as slow as the large one.

4.5 BANDIT POLICY AND EXPLORATION STRENGTH

UCB is one choice among standard bandit policies. Figure 5 compares it against ϵ -greedy and Thompson sampling across their exploration parameters, scoring each by the fraction of the better-versus-worse backend gap it captures at $\alpha = 0$. Sweeping three policies over five settings on three benchmarks live is not affordable, so this ablation is run by offline replay over the collected task-

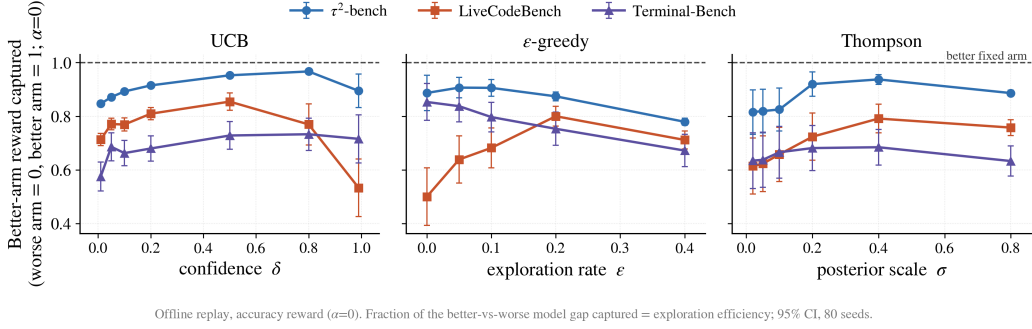


Figure 5: **Bandit policy and exploration parameter.** Fraction of the better-versus-worse backend reward gap captured under the accuracy reward ($\alpha = 0$), where 0 is always selecting the worse arm and 1 the better one. Error bars are 95% CIs over 80 seeds. UCB is the only policy whose optimum is stable across benchmarks.

outcome records with 80 seeds per configuration; on matched configurations, replay agrees with live execution to within 1–3 points of task accuracy, an interval well below the effects reported here.

The comparison favors UCB, but not primarily on peak performance. At each policy’s best setting UCB captures 0.968 / 0.855 / 0.735 of the gap on τ^2 , LiveCodeBench, and Terminal-Bench, against Thompson’s 0.94 / 0.79 / 0.685 at $\sigma \approx 0.4$ —an advantage, but a small one. The decisive difference is stability. UCB peaks in the band $\delta \in [0.5, 0.8]$ on all three benchmarks, whereas ϵ -greedy’s optimum moves from $\epsilon = 0.2$ on LiveCodeBench to $\epsilon = 0$ on Terminal-Bench, and the orderings cross: $\epsilon = 0$ is simultaneously the highest Terminal-Bench configuration in the figure (0.855) and the lowest LiveCodeBench one (0.50). A single ϵ chosen without workload knowledge is therefore as likely to be the worst available setting as the best—knowledge an online router does not have at admission. Thompson sampling is stable in optimum but exhibits wide confidence intervals at small σ , where the posterior is too tight to explore. At the deployed $\delta = 0.1$ UCB captures 0.89 / 0.77 / 0.66 against 0.95 / 0.86 / 0.73 at $\delta = 0.5$, so the frontiers of subsection 4.2 are obtained at a conservative setting and understate what the policy attains at its preferred one.

4.6 SCALING BEYOND TWO CANDIDATE MODELS

We extend our experiments to include two additional candidate models and study the impact of the routing choice made by TRACE-Router. Beyond the Qwen3.5-9B (model A) and Qwen3.5-4B (model B), we add Gemma-12B (model C) and Deepseek-R1-Distill-14B (model D) as the other two LLM backends.

4.6.1 ROUTING τ^2 Bench WITH FOUR CANDIDATE MODELS

The first plot in fig. 6 shows that TRACE-Router outperforms both the semantic and complexity routers in the telecom domain. During exploration, the learns to favor Qwen3.5-9B, achieving the best accuracy–latency trade-off with an accuracy of 31.24% and an average latency of approximately 12 s. In contrast, the semantic router predominantly selects DeepSeek-R1, while the complexity router consistently routes tasks to Gemma-12B. Both models frequently exceed the 150 s timeout, resulting in lower accuracy and significantly higher latency.

The second plot in fig. 6 shows that TRACE-Router achieves substantially lower latency than both the semantic and complexity routers, with the best configuration attaining an average latency of 18 s. This improvement stems from the reward function favoring the low-latency Qwen3.5-4B (Model B), even at the expense of the higher accuracy offered by Qwen3.5-9B (Model A). In contrast, the semantic router primarily selects a combination of Models A and D, increasing the accuracy to 61.3% but also raising the average latency to 36 s. Meanwhile, the complexity router consistently routes requests to Model C, resulting in a substantially lower accuracy of just 9.1%.

The key takeaway from this four-model ablation study is that TRACE-Router consistently outperforms semantic and complexity-based routing by leveraging inter-task feedback to learn better rout-

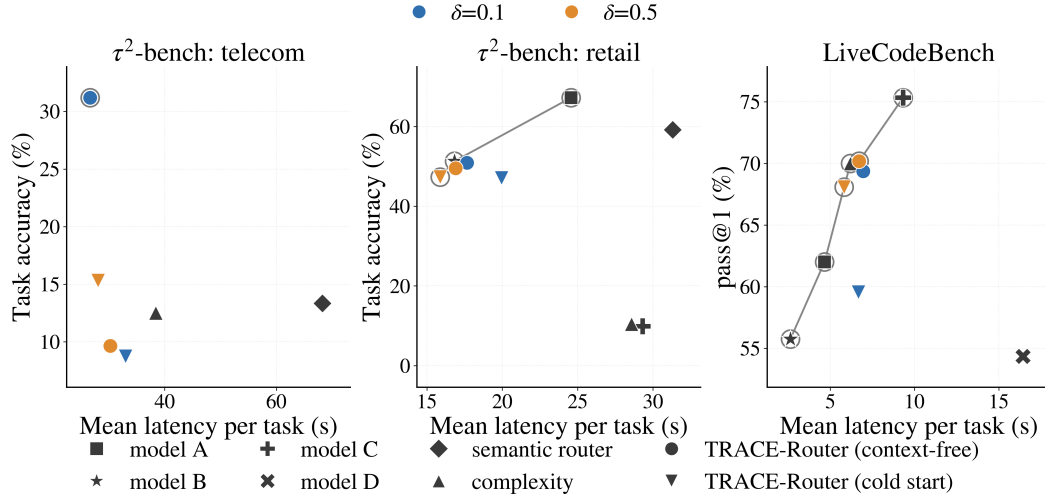


Figure 6: **Routing with four candidate models:** Accuracy vs. latency comparison for different routers running τ^2 Bench (telecom and retail domain) and LiveCodeBench. TRACE-Router uses $\delta = 0.1$ (blue) and $\delta = 0.5$ (orange) for context-free and cold-start configurations.

ing decisions. Unlike the complexity router, which cannot adapt to changing task distributions, TRACE-Router quickly identifies unreliable or timeout-prone models and shifts requests to better-performing alternatives, resulting in higher overall reward.

4.6.2 ROUTING *LiveCodeBench* WITH FOUR CANDIDATE MODELS

The third plot in Figure 6 compares TRACE-Router with the individual models, the semantic router, and the complexity router. Among the individual models, Gemma-12B achieves the highest accuracy (75.3%), while Qwen3.5-4B provides the lowest latency (2.8 s). The complexity router obtains the highest reward by primarily selecting the high-accuracy Gemma-12B and Qwen3.5-9B. In contrast, TRACE-Router favors the lower-latency Qwen3.5-4B, sacrificing some accuracy to improve the overall latency-reward trade-off. Among the bandit variants, the contextual bandit with $\delta = 0.5$ achieves the best performance.

Overall, these results demonstrate that bandit-based routing effectively explores the candidate model space and learns an adaptive routing policy that balances accuracy and latency. As the number of candidate models increases, exhaustive manual tuning becomes impractical, making adaptive exploration and feedback-driven model selection essential for identifying the optimal model for each task.

5 CONCLUSION

We introduced TRACE-Router, an online LLM router that commits a single model to an entire agentic task trace rather than routing each step independently, and learns this policy from bandit feedback via a contextual bandit. By aligning the routing decision with the granularity at which agentic work actually unfolds, our approach preserves within-trace consistency while adapting to observed reward, and our experiments show that it matches or improves end-to-end task quality at lower cost than per-query routing and static baselines. TRACE-Router shows that task-consistent commitment is a practical unit for deployment-time model selection, and a promising next step is extending it to hierarchical, multi-agent orchestration under distribution shift.

REFERENCES

Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, et al. Automix:

-
- Automatically mixing language models. *Advances in Neural Information Processing Systems*, 37:131000–131034, 2024.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.
- Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*, 2024a. URL <https://openreview.net/forum?id=cSimKw5p6R>.
- Shuhao Chen, Weisen Jiang, Baijiong Lin, James Kwok, and Yu Zhang. Routerdc: Query-based router by dual contrastive learning for assembling large language models. *Advances in Neural Information Processing Systems*, 37:66305–66328, 2024b.
- Tao Feng, Yanzhen Shen, and Jiaxuan You. Graphrouter: A graph-based router for llm selections. In *International Conference on Learning Representations*, volume 2025, pp. 26186–26203, 2025.
- Neel Guha, Mayee F Chen, Trevor Chow, Ishan S Khare, and Christopher Re. Smoothie: Label free language model routing. *Advances in Neural Information Processing Systems*, 37:127645–127672, 2024.
- Canbin Huang, Tianyuan Shi, Yuhua Zhu, Ruijun Chen, and Xiaojun Quan. Lookahead routing for large language models. *Advances in Neural Information Processing Systems*, 38:59042–59067, 2026.
- Hao Kang, Ziyang Li, Xinyu Yang, Weili Xu, Yinfang Chen, Junxiong Wang, Beidi Chen, Tushar Krishna, Chenfeng Xu, and Simran Arora. Thunderagent: A simple, fast and program-aware agentic inference system. *arXiv preprint arXiv:2602.13692*, 2026.
- Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E Gonzalez, et al. Autellix: An efficient serving engine for llm agents as general programs. *arXiv preprint arXiv:2502.13965*, 2025.
- Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. Routellm: Learning to route llms with preference data, 2024. URL <https://arxiv.org/abs/2406.18665>.
- Seongho Son, Sangwoong Yoon, Jiahua Tang, Shuhan Wang, Lorenz Wolf, and Ilija Bogunovic. Swe-router: Routing in multi-turn agentic software engineering tasks. *arXiv preprint arXiv:2607.00053*, 2026.
- Jiayi Tian, Yupeng Su, Ryan Solgi, Souvik Kundu, and Zheng Zhang. Rankguide: Tensor-rank-guided routing and steering for efficient reasoning. *arXiv preprint arXiv:2604.16694*, 2026.
- Asterios Tsourvas, Wei Sun, and Georgia Perakis. Causal llm routing: End-to-end regret minimization from observational data. *Advances in Neural Information Processing Systems*, 38:37289–37314, 2026.
- Noppanat Wadlom, Junyi Shen, and Yao Lu. Efficient llm serving for agentic workflows: A data systems perspective. *Proceedings of the ACM on Management of Data*, 4(3 (SIGMOD)):1–29, 2026.
- Marcel Wagenländer, Otto White, Britannio Jarrett, Pedro Silvestre, Yanda Tao, Guo Li, Huanzhou Zhu, Lluis Vilanova, and Peter Pietzuch. Sceptsy: Serving agentic workflows using aggregate llm pipelines. *arXiv preprint arXiv:2604.15186*, 2026.
- Jiaqi Xue, Qian Lou, Jiarong Xing, and Heng Huang. R2-router: A new paradigm for llm routing with reasoning, 2026. URL <https://arxiv.org/abs/2602.02823>. Accepted to ICML 2026.
- Jiarui Zhang, Xiangyu Liu, Yong Hu, Chaoyue Niu, Fan Wu, and Guihai Chen. Ragrouter: Learning to route queries to multiple retrieval-augmented language models. *Advances in Neural Information Processing Systems*, 38:15709–15737, 2026.

A PRIOR-WARM-STARTED CONTEXTUAL UCB

This appendix specifies the prior-warm-started variant of the bandit policy of subsection 3.3, evaluated as an ablation in subsection 4.4. It replaces the zero-initialization of the cold-start policy with pseudo-observations drawn from a static prior, so that the first routing decision in each context follows a supplied preference rather than a round-robin forced pull.

Let $h : \mathcal{C} \rightarrow \mathcal{M}$ be a static prior policy that names a preferred backend $h(c)$ for each context c , and let $\mu_{c,m}^0$ be a prior mean reward assigned to backend m in context c , expressed on the same scale as the task reward $r_{t,m}^{(\alpha)}$ of Equation 11. For every context-backend pair the bandit statistics of subsection 3.3 are initialized not to zero but to

$$N_{c,m}^{(0)} = w, \quad S_{c,m}^{(0)} = w \mu_{c,m}^0, \quad (16)$$

where the pseudo-count $w > 0$ sets how many synthetic observations the prior is worth. All backends receive the same pseudo-count w , while the prior-preferred backend is given a strictly larger prior mean,

$$\mu_{c,h(c)}^0 > \mu_{c,m}^0 \quad \text{for all } m \neq h(c), \quad (17)$$

so that it is the initial $\arg \max$ of the selection rule in Equation 9.

Because all backends begin with equal pseudo-counts, their exploration bonuses in Equation 9 are initially equal, and the first routing decision is governed by the prior means in Equation 16 rather than by an unbounded exploration term; no forced pulls are required. Real observations then accumulate on top of the pseudo-statistics. After backend m has served n completed tasks in context c , returning rewards $r_1, \dots, r_n$, its value estimate from Equation 8 becomes the pseudo-count-weighted mean

$$\hat{\mu}_{c,m}^{(n)} = \frac{w \mu_{c,m}^0 + \sum_{i=1}^n r_i}{w + n}, \quad (18)$$

in which the prior contributes a fraction $w/(w+n)$ of the estimate. This fraction decreases as evidence accumulates, so a reliable prior improves early routing while sufficient online feedback can overturn an incorrect one. The pseudo-count w interpolates between static and fully online routing: as $w \rightarrow 0$ the prior’s weight $w/(w+n)$ vanishes after a single observation, so warm start approaches the cold-start policy of subsection 3.3 in behavior, differing only in that cold start reaches a defined estimate through forced pulls (Equation 7) rather than through a prior.

Since the prior contributes only finitely many pseudo-observations, its influence vanishes as n grows: under stationary assumptions the warm-started and cold-started policies target the same reward-optimal backend within each context. The two variants therefore differ only in transient behavior—precisely the regime a finite task stream occupies. subsection 4.4 measures that regime and finds it favors cold start, which is why the warm-started policy is reported here as an ablation rather than as TRACE-Router’s default.